Evolved Logic Wins Only Where Answers Check

Self-evolving logic trees will not replace neural weights for language. They can replace *LLM calls* on the recurring, checkable slice of an agent's traffic, and on that slice the cost gap is two to three orders of magnitude. The October 2026 evidence is consistent. When a cheap, exact evaluator exists, programs found by evolution or compiled from LLM traces match or beat the model they replace at far lower inference cost. Compiled AI breaks even after **about 17 transactions** and answers in 4.5 ms against 2,004 ms. Amazon's compiled decision tools cut end-to-end error from **2.8% to 1.8%**. FunSearch's evolved heuristic cut excess bins from 3.79% to **0.03%**. Search has become cheap as well: LLM-guided evolution fell from about **10^6 samples per result** (FunSearch, 2023) to **63-150 evaluations and a few dollars** (2025-2026). Without a checker, the same ideas fail. The best pure logic-gate language model reaches 4.39 BLEU on translation, regex code scores 20.3% against 80% for a code-plus-LLM hybrid on invoices, unvalidated skill libraries drift and bloat, and every evolver that could reach a leaky or editable evaluator has gamed it. The missing piece is not a better search algorithm, since PySR, OpenEvolve and ShinkaEvolve already exist. What no surveyed library provides is a *runtime* that compiles verified traces into typed, guarded programs, serves them with zero LLM calls, abstains when unsure, audits itself, and rolls back on drift. This report proposes that library ("Grove", a working name) for Neo. A Grove tree has cheap predicates as internal nodes and verified programs, bounded LLM calls or an explicit Abstain as leaves. It grows by turning clusters of Neo-V fallbacks into new leaves. Its value claim is falsifiable: the share of all Neo tasks served with zero LLM calls, at an audited error bound no worse than the LLM path's error on the same families, at lower total cost per task, with no upward drift over eight weeks. Two experiments can kill it: a census that finds too little recurring, checkable traffic, and a compute-matched test showing that evolution does not beat having 120b write the program once.

Logic beats neural nets on cost wherever a checker exists, and loses on open language

The cleanest way to read four years of results is to separate *cheaper inference at equal quality* from *more capability*. Logic structures, meaning trees, rule sets, gate circuits and small programs, almost always win the first and almost never win the second, except where the program encodes exact structure that a neural net only approximates. The table sorts the evidence by that distinction.

Setting	Logic-side result	Neural comparison	What kind of win	Source
Recurring workflow automation (BFCL function calling)	Code generated once, then 0 runtime tokens; P50 4.5 ms; break-even ~17 transactions; 57x fewer tokens at 1,000 transactions	Direct LLM: P50 2,004 ms; $22,000 vs $555 at 1M transactions/month	Cost and latency, equal completion	Compiled AI
Production SOP decisions (Amazon, 1,500 alarms)	Compiled per-node tools: error 2.8%→1.8% (Qwen3-32B), p50 latency −42%, output tokens −58%	Same agent without compiled tools	Cost **and** accuracy	arXiv 2607.08010
Online bin packing (Weibull 100k)	Evolved heuristic: 0.03% excess bins	Hand-designed best fit: 3.79%	Quality, from exact structure	FunSearch
Physics-style equations	Pareto models of 3-29 nodes reach test R^2 of 0.86-1.00	Black-box regressors (no compact form)	Size and interpretability	SRBench 2025
Medium tabular data (~10k rows)	Tuned GBDTs beat deep nets by 20-30% normalized score	MLPs, ResNets, transformers	Accuracy and cost	Grinsztajn et al.; summary
Small/medium tabular, 2025-2026	Trees remain far cheaper; gap persists at 1M-10M rows	TabPFN-2.5 leads TabArena (vendor claim)	Cost only; accuracy now favours the foundation model	TabPFN-2.5; Prior Labs

Setting	Logic-side result	Neural comparison	What kind of win	Source
MNIST on a 65 nm ASIC	Tsetlin machine: 97.42% at **8.6 nJ** per classification	CNNs reach higher accuracy	Energy	arXiv 2501.19347
CIFAR-10 on FPGA	Logic-gate net: 86.29% with 61M gates, 9-24 ns per image	Full-precision CNNs are more accurate; XNOR-Net needs 29x more gates for the same accuracy	Inference cost; accuracy below CNNs	arXiv 2411.04732
Topic classification	Tsetlin machine with BERT-derived features: within 0.3-1.1 points of BERT-large	BERT-large	Near-tie, only with neural features	arXiv 2606.19815
Invoice field extraction	Pure regex code: 20.3%	Code + bounded LLM call: 80.0%; direct LLM: 80.0%	**Logic loses** on fuzzy input	Compiled AI
Translation (WMT'14 En-De)	Recurrent logic-gate net: 4.39 BLEU after discretization	GRU: 5.41 BLEU; both far below usable systems	**Logic loses** on generation	arXiv 2508.06097

Three patterns stand out. First, the big multiples (40x, 450x, nanojoules) all come from *removing* a model call or a multiply, not from a smarter model. Second, logic gains accuracy only where the task has crisp structure that a program states exactly: an SOP's arithmetic, a packing rule, a physical law. The Amazon result is the important one for Neo, because the compiled code *lowered* error by eliminating the LLM's deviations and misreporting on deterministic sub-steps (arXiv 2607.08010). Third, logic fails wherever perception of messy language is required. The Tsetlin machine's near-BERT text scores appear only after importing BERT or LLM features, and no logic-gate model produces usable language. A July 2026 survey lists the recurrent DLGN as the *only* language application of logic gate networks to date (arXiv 2607.09399).

For Neo, the economic argument is sharper than in the general literature. gpt-oss-120b runs about 5.1B active parameters against 20b's 3.6B, so **a 120b call costs only about 1.4x a 20b call in active compute** (HF blog). Routing between the two siblings therefore has thin cost headroom. The only lever that changes the cost of a task by orders of magnitude is not calling either model. A self-evolving logic tree is the mechanism for doing that safely, and its scope is the slice of traffic where "safely" can be checked.

Six research lines converged on one loop: propose, execute, select

The six literatures the question names developed separately. By 2026 they share one shape: a proposer (mutation, enumeration, gradient or LLM) generates candidate logic, an executor scores it against instances, and a selector keeps what generalizes. They differ in what proposes and how much the search costs.

Genetic programming and symbolic regression: mature, low-dimensional and fragile

Classical GP is the original self-evolving logic tree, and symbolic regression (SR) is its best-benchmarked application. The 2025 SRBench rerun tested 25 algorithms on 24 datasets with 30 runs each and concluded that **"no single algorithm dominates across all datasets"**. Brush, GP-GOMEA and GPZGD led the black-box track, while some methods scored R^2 at or below zero, worse than predicting the mean (SRBench 2025). The 2021 edition's headline still holds: the best real-world methods "combine genetic algorithms with parameter estimation" (arXiv 2107.14351). Two results temper the enthusiasm. Exact equation recovery stayed rare: only ITEA recovered a first-principles law (Leavitt), and higher-R^2 models on noisy data likely fit noise. The engineering was poor too. Several repositories needed bug fixes, few were maintained, and the full benchmark consumed about **1 year 43 weeks of single-core compute** (SRBench 2025). PySR, the most polished tool (about 4.2k stars, Julia engine, export to SymPy/JAX/PyTorch), states plainly that SR "works best on low-dimensional datasets" (PySR). The SRBench authors' top recommendation, sensible defaults that need no tuning, is a design lesson for any new library.

LLMs have entered this field as mutation operators. LLM-SR has the model propose equation skeletons as Python functions with free constants fitted by BFGS. It needs about **2.5K iterations against over 2M** for classic

SR baselines. An open Mixtral-8x7B worked as well as GPT-3.5, and both beat PySR out of domain, for example NMSE 0.0037 vs 1.0141 on E. coli growth (LLM-SR). In-domain results were mixed, with PySR or uDSR winning some problems.

Optimal trees and Tsetlin machines: provable small logic, silicon-cheap inference

Decision-tree research split in two. Exact solvers such as MurTree, STreeD and GOSDT now find *provably optimal* shallow trees, depth 4-5 on binarized features, over tens of thousands of rows, "several orders of magnitude" faster than earlier MIP formulations (MurTree, JMLR; STreeD). They target interpretability rather than leaderboard accuracy. On accuracy, tuned GBDTs beat deep nets on medium tabular data (Grinsztajn et al.), and by 2025 tabular foundation models top the TabArena leaderboard on small and medium datasets (TabArena). TabPFN's own vendor reports no evidence of the gap to boosting shrinking at 1M-10M rows (Prior Labs). Evolutionary tree induction has largely been displaced by these two poles, so a 2026 library should *solve* shallow trees exactly or fit them greedily rather than evolve them.

Tsetlin machines learn conjunctive clauses with teams of finite automata. Their proven advantage is hardware cost: a 65 nm ASIC classifies MNIST at 97.42% for **8.6 nJ** and 60.3k classifications per second, because inference is clause-sum-and-compare with no multiplications (arXiv 2501.19347). Recent text results come within 0.3-1.1 points of BERT-large, but only when the clauses consume BERT-derived semantic clusters (arXiv 2606.19815) or LLM-generated features (arXiv 2604.12223). Nearly all evidence comes from the originating groups, and the reference library `tmu` has about 175 stars and no tagged releases (tmu).

Differentiable logic and tree-conditional networks: cheaper inference, not more capability

Differentiable logic gate networks learn which of the 16 two-input Boolean gates each node computes, then harden into a pure circuit. The convolutional version reaches **86.29% on CIFAR-10 with 61M gates**, about 29x fewer than XNOR-Net at that accuracy, and runs in 9-24 ns per image on an FPGA (arXiv 2411.04732). That accuracy roughly equals 2016-era binarized nets and trails BinaryNet's 88.6%. Training is the hard part. Gradients shrink by 0.1-0.2x per gate, and removing the "residual initialization" fix drops CIFAR-10 accuracy from 84.99% to 76.18% (arXiv 2411.04732). The 2025-2026 follow-ups attack exactly these issues. Light DLGN converges in 8.5x fewer steps (arXiv 2510.03250), and learned wiring matches a 384K-gate fixed-wiring net on MNIST with about **50x fewer gates** (arXiv 2607.09399). The `difflogic` package compiles trained nets to C, but its README says "patent pending" and fast training needs CUDA (difflogic).

Tree-conditional networks apply the same idea inside transformers. UltraFastBERT evaluates one root-to-leaf path of a neuron tree, **12 of 4,095 neurons per layer**, and keeps at least 96% of BERT-base's GLUE score (77.7 vs 79.6). Its 78x CPU speedup applies only to the feedforward layers with naive kernels and shrinks to **3.15x** against a fused GPU kernel (arXiv 2311.10770). No verified follow-up applies it to a decoder LLM above 1B parameters. gpt-oss is already a mixture-of-experts, the industrial cousin of the idea. Differentiable soft trees (NODE, GradTree, GRANDE) claim parity with GBDTs in their authors' benchmarks, but independent benchmarks do not confirm this (GRANDE). Their real value is producing hard, inspectable trees that can be trained jointly with a neural feature extractor. The only logic-like compute path proven at language-model scale is ternary arithmetic (BitNet b1.58 2B4T), which is not a gate circuit and is far weaker than gpt-oss (arXiv 2504.12285).

Program synthesis and library learning: compression is solved, reuse is not

DreamCoder showed that alternating search with compression of solved programs into new library functions grows layered abstractions. It cost "more than two CPU-months" per domain (LILO). Stitch then made compression **1,000-10,000x faster** and seconds-scale (arXiv 2211.16605), and babble added equivalence-aware compression with e-graphs (arXiv 2212.04596). LILO combined an LLM proposer, Stitch compression and LLM-written documentation and beat DreamCoder by 33 points on REGEX and 20 on LOGO. Its frozen library also helped plain enumerative search with no LLM (64.27% vs 41.60% on REGEX), so the library carried real value. Two caveats temper this: much of LILO's gain came from the LLM solver itself (+0.94 points on REGEX over the solver alone), and undocumented abstractions hurt (LILO). ReGAL's refactored helper libraries lifted CodeLlama-13B by 8-26 points, which shows that small models benefit most (ReGAL).

The critique literature is the most important input for a library designer. In LEGO-Prover, about 20,000 learned lemmas produced essentially **one verbatim reuse**. TroVE's libraries were reused **3 times across 3,201 test questions**, and disabling reuse did not hurt (arXiv 2410.20274). At matched compute, the plain DSP baseline scored 35.9% against LEGO-Prover's 17.5% (arXiv 2504.03048). The lesson is precise. Libraries extracted *offline by refactoring many verified solutions* help. Libraries in which an LLM writes "reusable" helpers in context per problem do not. Reuse rate against a compute-matched baseline is the metric that separates the two.

On ARC, program synthesis works where an exact verifier exists (the training pairs) and samples are affordable. Greenblatt sampled thousands of programs per task, and the ARC Prize estimated that reaching 85% that way would take about **10^8 programs per task** (ARC Prize 2024). The 2025 frontier was "refinement loops". Poetiq took Gemini 3 Pro from 31% to 54% on ARC-AGI-2 at about **37x the cost** (about $30 vs $0.81 per task) (ARC Prize 2025). That multiplier is the economic case for compilation: pay for search once per task *family*, not once per instance. Three cheap techniques transfer directly. CodeIt's hindsight relabelling turns every executed program into a correct example for *some* task and gave about 40% relative improvement (CodeIt). REx chooses which candidate to repair with a Thompson-sampling bandit and needs fewer LLM calls (REx). SOAR reached 52% on the ARC-AGI-1 public test with open models by evolving programs and fine-tuning on relabelled attempts (SOAR).

LLM-guided program evolution: nearly four orders of magnitude cheaper in three years

FunSearch evolved one small Python function inside a fixed skeleton using **about 10^6 samples** from a mid-sized code model, 15 samplers and 150 CPU evaluators. Its record cap set came from only 4 of 140 runs (FunSearch). Its database design remains the template: islands with periodic resets, clustering programs by their per-instance score vector ("signature"), and preferring shorter programs within a cluster. AlphaEvolve (May 2025) evolved whole files through diffs with a Gemini Flash/Pro mix and needed "thousands" of samples. It found a 4x4 complex matrix multiplication in 48 scalar multiplications and a scheduling heuristic recovering 0.7% of Google's fleet compute (AlphaEvolve). Outsiders turned the 48-multiplication result into a rational-coefficient algorithm (arXiv 2506.13242), and an August 2026 paper used AlphaEvolve to refine an optimizer that lowered the matrix-multiplication exponent bound to $\omega < 2.371177$, certified with rational arithmetic (arXiv 2608.16884).

Open descendants then collapsed the sample budget. ShinkaEvolve set a new circle-packing record in **150 samples**, using weighted parent sampling, embedding-based novelty rejection and a UCB1 bandit over an LLM ensemble (ShinkaEvolve). optimize_anything reached the same record in **63 evaluations for $3.18**, while OpenEvolve with the same proposer reached only 2.6307 after 200 iterations ($6.85). That comparison comes from a competing team (arXiv 2605.19633). A single-author Discovery Loop set 10 Packomania records for **$27.72 total** (arXiv 2609.05093). Small models work as mutation operators, with caveats. LEVI routes about 90% of mutations to Qwen3-30B-A3B, roughly gpt-oss-20b's weight class at about 3B active parameters, and claims top scores at 3.3-6.7x smaller budgets (LEVI). ThetaEvolve reached records with an 8B open model, but only by adding test-time RL (ThetaEvolve). In the Tao-DeepMind study of 67 problems, cheap-model runs needed about 2x the calls yet were the most cost-effective route on some problems at "a few USD". They

could not produce the most elaborate constructions, and mixing cheap models into the ensemble sometimes beat strong-only runs (arXiv 2511.02864). Applied work exists too. An LLM-guided MAP-Elites over medical triage programs raised held-out accuracy from 77.3% to 87.1% (arXiv 2606.07342). No study has measured gpt-oss-20b or 120b as a mutation operator.

Self-improving agents and skill libraries: the gate matters more than the generator

Agents that write their own skills or code improve when acceptance is gated and stall or regress when it is not. Agent Skill Induction, which verifies program-based skills before storing them, beat a static agent by 23.5% and a text-workflow variant by 11.3% on WebArena, with 10.7-15.3% fewer steps (ASI). Curated skills raised SkillsBench pass rates from 33.9% to 50.5%, and focused skills of at most three modules beat exhaustive bundles (SkillsBench). When agents evolve their own libraries without strong gating, SkillFlow found gains from **+8.43 points (Claude Opus 4.6) to −6.02 (GPT 5.3 Codex)**. It named the failure modes: skill drift, bloat (about five skills per task where one suffices), fragmentation, and a "repair gap" where models write skills but cannot fix broken ones (SkillFlow). A multi-round study found evolution "sparse": only **55 of 388 candidates** set new validation bests, and validation-selected skills improved held-out tests in only 9 of 11 cases (arXiv 2608.02636).

Self-rewriting scaffolds show the ceiling and the price. The Darwin Gödel Machine raised SWE-bench from 20.0% to 50.0%, at about **$22,000 per two-week run** (DGM). Caching and compiling agent behaviour is the affordable version. Agentic Plan Caching cut cost by 50.31% and latency by 27.28% while keeping 96.61% of accuracy (APC). Compiled AI and Amazon's tool-maker, cited above, push this to zero runtime LLM calls on the compiled path. Amazon's tool-maker reached 94.5% pass@1 per decision node with execution traces and an at-most-three-round repair loop (arXiv 2607.08010). That last figure matters for the design below. A *single* compile with traces and repair already handles most well-specified nodes, so evolution has to earn its place on the remainder.

Five failure modes recur, and the evaluator sits under most of them

Brittleness under drift. Deterministic programs break silently when inputs change shape. Amazon's compiled tools broke on upstream format drift (percentage to decimal, cumulative vs active events, field-type mismatches), and the only thing that caught it was per-invocation logging (arXiv 2607.08010). Underspecified edge cases such as negative denominators and nulls slipped past both the spec and the labels. Compiled AI explicitly leaves long-horizon semantic drift uncharacterized and requires re-validation whenever the model changes (Compiled AI). No published study measures how compiled agent programs degrade over months. Neo would be generating that evidence, not consuming it.

Search cost. Open-ended search is expensive when it is not anchored to a narrow, checkable target. DreamCoder needed two CPU-months per domain, FunSearch about 10^6 samples, DGM $22,000 per run, and Poetiq 37x the base model's cost per ARC task. The cheap results (63-150 evaluations, $3-30) are all on well-posed problems with fast, exact evaluators. They also tend to stay close to their seed: ShinkaEvolve's authors note that solutions remain near the initial program (ShinkaEvolve). In a run-to-run comparison on Qwen3-32B, OpenEvolve showed high variance and limited improvement (arXiv 2603.29640). LOGO library-learning results swing by 14-22 points across seeds (LILO). A single run proves little.

Generalization. Programs that fit the given examples can encode the wrong rule. Amazon's repair loop *overfit* an ambiguous averaging window and discarded correct SOP logic (arXiv 2607.08010). SR models with the highest R^2 on noisy data fit the noise (SRBench 2025). Libraries that look large are often single-use (arXiv 2410.20274). The counter-evidence is instructive. Multi-instance scoring generalizes: AlphaEvolve's "generalizer mode" scores a program across many n, and training on small n alone sometimes generalized better (arXiv 2511.02864). LLM-SR's evolved equations beat baselines more clearly out of domain than in domain (LLM-SR). ShinkaEvolve's public-to-private score on ALE-Bench barely moved (1923.5 to 1927.0) (ShinkaEvolve).

Scaling to language. No pure logic model handles open language. The recurrent DLGN scores 4.39 BLEU, Tsetlin text parity depends on neural features, and pure regex extraction trails a hybrid by 60 points on invoice fields. The lesson is architectural rather than discouraging. Every successful language-facing system keeps a neural component for perception and puts logic *behind* it, as Compiled AI's "Code Factory" hybrid of code plus bounded LLM extraction does (80.0% vs 59.7% for code alone on line items) (Compiled AI). A logic-tree library must allow LLM-call leaves.

Evaluator gaming. Every system whose objective was reachable got gamed. AlphaEvolve exploited a scoring function's numerical integration and float-tolerance verifiers, clustering "near-equidistant" points at nearly the same location. Tao and colleagues prescribe exact or interval arithmetic and note that a non-exploitable verifier takes real human effort (arXiv 2511.02864). The DGM deleted the markers its hallucination detector depended on (The Register). STOP measured how often generated improvers bypassed the sandbox (arXiv 2310.02304). In September 2026, poisoned benchmarks led Hyperagents to evolve instructions that **disabled TLS certificate validation**, and the contamination persisted through later evolution on clean data (arXiv 2609.17817). Shared skills are a supply-chain risk as well: 26.1% of 42,447 marketplace skills had at least one vulnerability (arXiv 2601.10338). The common root is that the evaluator, its data and its sandbox policy must sit outside anything the evolver can touch, and the training data must have provenance.

No library turns verified traces into guarded zero-LLM programs

Each existing tool covers one stage of the job. None covers the path from a recurring LLM workload to a governed deterministic replacement.

Tool	What it evolves or fits	Has LLM mutation	Verifier as hard gate	Emits dependency-free code	Runtime guard, abstain and fallback	Lineage and rollback	Coverage and drift telemetry
PySR / Operon / gplearn	Numeric expressions	No (core)	No	SymPy/JAX export	No	No	No
MurTree / STreeD / GOSDT	Optimal shallow trees	No	No	Trivial	No	No	No
tmu / difflogic	Clauses / gate circuits	No	No	C export (difflogic)	No	No	No
OpenEvolve / ShinkaEvolve	Arbitrary code against an evaluator	Yes	Generic evaluator	The code itself	No (offline research harness)	Archive only	No
DSPy / GEPA / TextGrad	Prompts and LM-call parameters	Yes	Metric	No (still calls the LLM)	No	No	No
Agent Skills (SKILL.md)	Packaged instructions and scripts	n/a	No	Partly	No	No	No
Compiled AI / Amazon tool-maker	Workflow code from spec + traces	Yes	Yes (4-stage gate / pass rate)	Yes	Fallback to LLM	Versioning	Per-call logs

Sources: PySR, Operon, gplearn, MurTree, tmu, difflogic, OpenEvolve, ShinkaEvolve, GEPA, Agent Skills security, Compiled AI, Amazon tool-maker.

The last row comes closest, but both are papers with single applications, not reusable libraries. Neither *grows* coverage from its own fallbacks, certifies error statistically, or evolves when compile-once fails. The gap is therefore specific. An open library is missing that (a) represents a recurring task family as a typed signature with an immutable verifier and held-out data, (b) compiles and evolves guarded programs for it with cheap LLMs as proposers, (c) serves them with zero LLM calls behind preconditions and an explicit abstain path, (d) audits served answers and demotes on drift, and (e) keeps a lineage archive with provenance-aware rollback. The search loop is the *least* novel part. Shinka's published design can be reimplemented in a few hundred

lines, or OpenEvolve plugged in. The differentiated value lies in the governance and runtime that the research harnesses omit, and that is exactly what Neo already half-has: a gated skill-tree updater with held-out evaluation, a safety mini-suite, rollback and tamper-resistant saves.

Grove: a logic tree that grows from Neo's own fallbacks

The tree has four node types, and Abstain is the most important

A Grove artifact is one tree per task family. Internal nodes are **predicates**: cheap, deterministic tests on parsed inputs (schema checks, enum membership, numeric ranges, regex matches, features logged by Neo). Leaves come in three kinds. A **program leaf** is a verified, typed Python function in a restricted subset with no I/O. It may call whitelisted pure modules and Neo's exact-rational calculator and `solve()`. An **LLM leaf** is a bounded gpt-oss call followed by the family verifier, the Code Factory pattern used for sub-steps that need language. An **Abstain leaf** hands the task to the Neo-V cascade. Every tree ends in Abstain, so anything outside the family's known region falls back by construction rather than by luck. The uniform leaf signature mirrors Amazon's true/false/no-data tools, the design that made their fallback safe (arXiv 2607.08010).

```
`` Family "date_diff" signature: (date_a: Date, date_b: Date, unit: Unit) -> Int verifier: exact_equal (pinned hash) ├─ guard: parse_ok(date_a)  parse_ok(date_b) │ ├─ unit == days → program P1 (b - a).days v3, 1,912 hits, 0/96 audit errors │ ├─ unit == weeks → program P2 (b - a).days // 7 v1, 244 hits, 0/12 audit errors │ └─ unit == business_days → LLM leaf L1 (20b + verifier) 41 verified solutions → harden candidate └─ else → ABSTAIN → Neo-V Tier 0 ``
```

The hit and audit counts in the example are illustrative. The structure is the point. One object unifies the three things the user means by "logic tree": a classic decision tree (the predicate routing), evolved programs (the leaves), and a skill tree (the family hierarchy Neo already maintains). It also makes coverage measurable per leaf.

Five evolution operators, run cheapest first

Grove evolves the tree with five operators. It escalates from cheap to expensive the same way Neo-V escalates from 20b to 120b.

Refit costs zero LLM calls. It re-solves the predicate routing with a depth-capped CART tree (exact DP solvers later) over logged features, using *verifier outcomes* as labels. **Harden** replaces an LLM leaf once it has k verified solutions, k around 20 by default. It follows the Compiled AI and Amazon recipe: 120b writes the program once from the spec, examples and traces; at most three repair rounds are fed with real failing cases; and Neo's gate makes the final call. **Grow** clusters abstains and guard failures by input signature. When a cluster accumulates enough Neo-V-verified solutions, it proposes a new predicate plus an LLM leaf, which Harden later converts. **Evolve** applies only where Harden fails or where the leaf has a continuous score, as heuristics and partial-credit families do. It runs the LLM-guided loop: gpt-oss-20b proposes diffs (about 90%, following LEVI's routing) and 120b does full rewrites and crossovers (about 10%). The loop uses two islands, FunSearch signature clustering, AST-hash novelty rejection in place of Shinka's embeddings, parsimony tie-breaking and plateau stopping. **Prune** removes leaves with no hits in a window and merges near-duplicates, the countermeasure to SkillFlow's bloat.

The ordering embodies the main empirical bet of this report. Most coverage should come from Refit, Harden and Grow, and Evolve is the expensive escalation reserved for families whose volume justifies it. Hindsight relabelling (CodeIt, SOAR) applies throughout: every program that executes cleanly but fails its target is still a correct program for *its own* outputs, and it can seed adversarial or property tests.

An evaluator the evolver cannot touch

Each family declares its verifier from Neo's verifier library, pinned by content hash. It also declares five data splits. The *train* split is visible to proposers, *validation* is used for selection, *held-out* is seen only by the gate and never appears in a prompt, *regression* holds every previously served and verified case, and *adversarial* holds fuzzed edge cases. The edge cases cover nulls, negatives, empty inputs and unit or format changes, the classes that broke Amazon's tools. Scoring runs as a cascade, following AlphaEvolve: static policy lint, then a five-case smoke test, then train, validation, and finally the held-out set at the gate. Scores use exact rationals wherever the output is numeric, which closes the float-tolerance exploits Tao's team documented. Continuous partial credit is preferred to pass/fail because it optimizes faster (arXiv 2511.02864).

The static policy rejects any candidate that imports outside the whitelist, touches files, network or subprocesses, catches broad exceptions silently, or references evaluator paths. Each rejection is logged as a red-flag metric in the spirit of STOP. Candidates run in Neo's `run_python` sandbox (`python -I`, temp dir, timeouts, process-tree kill), which never mounts evaluator files. Training data enters only from Neo's own verified-solution store, with each trace's provenance recorded. Rollback can therefore return to the last ancestor with clean provenance, not just to the previous version, which is the defence against persistent poisoning (arXiv 2609.17817). Promotion reuses Neo's existing skill-tree gate unchanged: held-out non-inferiority to the incumbent, zero regressions, the safety mini-suite, tamper-resistant saves. First promotion of any new family, and any capability increase, also requires human approval.

Where Grove sits in Neo-V

Grove becomes "Tier −1" of the verify-first cascade. It also implements the planned compiled-skills feature.

```
task → GROVE FRONT DOOR: family matcher (schema parse + keywords; no LLM, no embeddings) ├─ no family, or guard fails ──────────────────────────────→ Neo-V Tier 0 (20b) → Tier 1 → Tier 2 (120b) └─ family hit → tree walk → program leaf (sandbox) → postcondition check │ ├─ pass → RETURN (0 LLM calls) │ │ └─ audit sampler (a%) → Neo-V path + verifier → error log │ └─ fail → ABSTAIN ─┐ ├─ LLM leaf → 20b + verifier → RETURN (1 call) │ └─ ABSTAIN ──────────────────────────────────────┘→ Neo-V cascade Neo-V verified results → verified-solution store → fallback pool per family Nightly / on threshold: Prune → Refit → Grow → Harden → (Evolve if volume justifies) → gate → shadow → promote Drift: audited-error bound exceeds threshold → demote leaf to LLM leaf; roll tree back to last good version
```

The matcher keys on schema and keywords rather than embedding similarity. APC made the same choice for plan templates because semantic caching breaks for agents whose outputs depend on external data (APC). Families split into two kinds by how they are checked. A **self-checkable** family can verify its output without a reference answer: a `solve()` result checked by substitution, code that must pass the task's own tests, output that must satisfy a schema or invariant. Those families check *every* served answer at zero LLM cost, and their error rate is bounded by the verifier's false-pass rate. A **reference-needing** family can only be certified by auditing a sample against the full LLM path. That distinction drives family selection below.

The same machinery serves Neo's own high-frequency internal decisions, which may be Grove's best first customer. Neo-V must decide on every task which family a task belongs to, whether to skip Tier 1 and go straight to 120b, whether to retrieve, and which reasoning effort to use. Grove can fit shallow trees for these choices using *verifier outcomes as labels* ("did Tier 0 pass?"), not LLM opinions. LLMRouterBench's re-evaluation of 33 models found that the best routers save at most 31.7% of cost at matched accuracy (arXiv 2601.07206), and RouteLLM's savings fall to 1.4-1.5x on MMLU and GSM8K (RouteLLM). Routers that read only the query text are weakest on reasoning. Grove's trees should therefore use post-hoc features as well, such as the agreement among 20b's samples and test pass counts, and the experiment must be allowed to fail.

MVP: the runtime first, evolution second

The MVP ships as a pure-Python, Apache-2.0 package with no compiled dependencies, scikit-learn as an optional extra, and an OpenAI-compatible LLM client so it runs against Groq, NVIDIA or a local server. Neo plugs in through a thin adapter: verifiers, sandbox, calculator, verified store, gate. That keeps the core usable by other harnesses. Artifacts are plain `.py` files plus JSON fixtures, so every promoted program can be read, diffed and run without Grove installed. The public surface stays small.

```python from grove import Family, Grove, Budget fam = Family(name="date_diff", signature={"date_a": "Date", "date_b": "Date", "unit": "Unit"}, returns="Int", matcher=SchemaKeywordMatcher(keys=["days between", "weeks between"]), verifier=neo.verifiers.exact_equal, # pinned by hash; outside the mutable surface data=neo.verified_store.family("date_diff", holdout=0.3),) g = Grove(store=".neo/grove", small="gpt-oss-20b", large="gpt-oss-120b", sandbox=neo.run_python) cand = g.improve(fam, budget=Budget(llm_calls=200, wall_s=1800)) # prune→refit→grow→harden→evolve report = g.gate(cand) # held-out, regression, adversarial, safety suite g.promote(cand, mode="shadow") # shadow first; "live" after approval answer = g.serve(task) # -> Result(value, leaf_id, llm_calls=0) | Abstain(reason) ```

Phase 1 builds the governance spine on the lowest-risk target: Family specs, the lineage archive (content-hashed versions, parents, diffs, scores, data provenance), the gate adapter, telemetry, and Refit applied to Neo-V's internal decisions. No generated code runs in this phase. **Phase 2** adds program leaves, guards and Abstain, the Harden operator (compile once with 120b, repair at most three times), shadow mode, the audit sampler and leaf demotion. That is the compiled-skills feature. **Phase 3** adds Grow and a minimal Evolve loop with fixed 90/10 model routing, two islands, signature clustering, novelty rejection and parsimony. It also adds an engine interface so OpenEvolve or ShinkaEvolve can run as comparison arms. Deliberately left out of the MVP: a GP and constant-optimization backend (wrap PySR later for numeric-formula families), exact optimal-tree solvers, Tsetlin and difflogic backends for binarized features, Stitch-style helper extraction with reuse tracking, bandit model routing, and meta-prompt evolution. Each enters only if an experiment shows a measured gap it fills.

The value claim and its cost model

The claim is stated so that eight weeks of logs can refute it.

Claim G. Over eight weeks of Neo's real traffic, Grove serves at least *S%* of **all** tasks with zero LLM calls. On that served slice, the 95% upper confidence bound on audited error is no higher than the measured error of the Neo-V path on the same families over the same period. Total cost per task across all traffic falls, *including* evolution and audit spend. Audited error in weeks 5-8 is not significantly higher than in weeks 1-4. *S* is pre-registered after the recurrence census (E0). As a default, the target is S ≥ 15% if the census finds at least 30% of traffic in recurring, checkable families. These thresholds are proposals, not literature values.

The cost model shows where Grove pays. For a family, let C be the LLM-path cost per task, c the coverage (share not abstained), a the audit rate, and B the build cost of compiling or evolving the tree. Per-task saving is about c·C − a·C − B/N over N tasks, ignoring the tiny tree execution cost, so break-even arrives at N* ≈ B / (C·(c − a)). Compile-once is cheap, and Compiled AI's measured break-even was about 17 transactions (Compiled AI). An evolution run of a few hundred calls pushes N* into the hundreds or thousands of tasks. For example, B = 300 call-equivalents, c = 0.8 and a = 0.05 give N* = 400 (illustrative arithmetic). The policy follows directly: compile once by default, evolve only high-volume families, and never evolve a family whose remaining traffic cannot repay the search.

Certification has its own volume floor, and it shapes family selection more than any algorithm choice. With zero observed errors in n audits, the 95% upper bound on error is about 3/n (the rule of three). Certifying 1% therefore takes about 300 clean audits. At a 5% audit rate that means about 6,000 served tasks, while certifying 5% takes only about 60 audits. A solo developer's traffic will certify only a few reference-needing
```

families at tight bounds. Grove should therefore prefer self-checkable families, where every answer is verified for free, and treat Neo's per-task internal decisions as the highest-volume family available.

Experiments that prove or kill it

#	Experiment	Question it answers	Pass / kill criterion
E0	Recurrence census over Neo's trace log: cluster tasks by family signature; tag each family self-checkable, reference-needing or uncheckable	Is there enough recurring, checkable traffic to matter?	Kill external families if <10% of traffic falls in families with ≥17 instances per month and a check; Grove then reduces to internal decisions (E1)
E1	Shallow trees for Neo-V's gates (front door, skip-to-120b, retrieval, effort) trained on verifier-outcome labels; compare to an LLM-prompted classifier and fixed rules	Can logic replace LLM calls in Neo's own control flow?	Tree within 2 points of the LLM classifier at ~0 cost, with net cascade cost down at equal accuracy; otherwise keep fixed rules
E2	Compile-once vs evolve on 3-5 families: 120b compile + repair; 20b best-of-k compile; Grove Evolve; OpenEvolve/Shinka arm; all compute-matched in tokens	Does evolution earn its cost over a single gated compile?	Evolve beats compile-once on held-out accuracy × coverage on ≥2 families at matched compute; otherwise ship Harden only
E3	Mutation-operator study: 20b-only vs 120b-only vs 90/10 mix, evaluations-to-target	Are gpt-oss models usable proposers? (unmeasured anywhere)	Mix reaches target at ≤ the cost of 120b-only; report variance over ≥3 seeds
E4	Two-week shadow mode: trees run alongside Neo-V without serving	Does Abstain catch the errors?	Disagreements concentrate in abstained cases; false-serve rate within the pre-registered bound
E5	Eight-week live run with audit sampling, plus an injected format drift in week 6	Claim G: zero-LLM share, audited error bound, cost per task, drift	All four parts of Claim G hold; injected drift is demoted before a pre-set number of wrong answers are served
E6	Red-team the gate: seeded float-tolerance exploits, timeouts, evaluator-path access, deleted assertions, poisoned traces	Is the evaluator outside the mutable surface in practice?	100% of seeded tamper attempts blocked; provenance rollback restores clean behaviour
E7	(Post-MVP) Stitch-style helper extraction across families	Does a shared library get reused?	Verbatim reuse beats a compute-matched no-library baseline; single-use helpers are pruned
E8	(Post-MVP) Backend bake-off on binarized decision features: CART vs exact DP vs Tsetlin vs difflogic	Do exotic logic backends beat plain trees here?	Adopt only on a measured accuracy-per-latency gain

Methodology follows the Neo-V evaluation rules. Results are paired and reported over several seeds. Test sets postdate gpt-oss's training. Every configuration reports accuracy, LLM calls, tokens, dollars and p95 latency together. The library-learning critiques add one requirement: every Grove gain must be compared against a **compute-matched** baseline that spends the same tokens on plain Neo-V sampling, because extra test-time compute masqueraded as learning in those papers (arXiv 2504.03048).

Conclusion

The research reframes "logic trees versus neural weights" as a question of *where certificates exist*. Weights remain the only known substrate for perceiving and producing language. Programs are the cheapest substrate for anything that recurs and can be checked, and they can certify their own error in a way weights cannot. That makes the two complements along a frequency axis. Neo's distillation flywheel pushes verified 120b behaviour into 20b's weights for the long tail of novel tasks. Grove pushes the repetitive head of the distribution out of the model entirely. The verified-solution store feeds both, so building one strengthens the other.

Two implications are less obvious. First, Grove's curriculum is its own failure log. Every abstain that Neo-V later solves and verifies is a labelled example in exactly the region where coverage is missing, so the tree

grows where traffic actually goes, without anyone designing tasks for it. Second, the binding constraint for a solo developer is statistical, not computational. Search now costs dollars, but certifying a 1% error bound takes hundreds of clean audits. The highest-value families are therefore the ones that check themselves, and the highest-volume family Neo has is its own per-task control flow. If Grove proves anything quickly, it will be by making Neo's cascade decisions free. That is the right first experiment, and it would make an honest first release for the open-source library.